CRUD Operations in SQL

(https://github.com/learn-co-curriculum/python-p3-sql-crud) (https://github.com/learn-co-curriculum/python-p3-sql-crud/issues/new)

Learning Goals

- Use SQL to store data and retrieve it later on.
- Use SQLite to build relational databases on your computer.
- Perform CRUD operations on relational databases using SQL.

Key Vocab

- **SQL (Structured Query Language)**: a programming language that is used to manage relational databases and perform operations on the data that they contain.
- **Relational Database**: a collection of data that is organized in well-defined relationships. The most common type of database.
- **Query**: a statement used to return data from a database.
- **Table**: a collection of related data in a database. Composed of rows and columns. Similar to a class in Python.
- **Row**: a single record in a database table. Each column represents an attribute of the record. Similar to an object in Python.
- **Column**: a single field in a database table. Each row contains values in each column. Similar to a Python object's attributes.
- **Schema**: a blueprint of the construction of the tables in a database and how they relate to one another.

Introduction

In this lesson, we'll cover different ways to manipulate and select data from SQL database tables. We'll see how to perform different Create, Read, Update, and Delete (or CRUD) actions in a database table.

Fork and clone this lesson (https://github.com/learn-co-curriculum/python-p3-sql-crud/fork) to follow along.

Setting Up Our Database

In this code along, we'll be working with a `cats` table in the provided `pets_database.db` file. Explore the `cats` table structure using the SQLite VSCode extension, or with DB Browser, or by running the `sqlite3` prompt:

```
$ sqlite3 pets_database.db
sqlite> .schema
CREATE TABLE cats (
  id INTEGER PRIMARY KEY,
  name TEXT,
  age INTEGER,
  breed TEXT
);
```

Okay, let's start storing some cats.

Code Along 1: INSERT INTO

Run the following command with the `pets_database.db` file (either in DB Browser, or in your terminal from the `sqlite3` prompt):

```
INSERT INTO cats (name, age, breed) VALUES ('Maru', 3, 'Scottish Fold');
```

We use the `INSERT INTO` command, followed by the name of the table to which we want to add data. Then, in parentheses, we put the column names that we will be filling with data. This is followed by the `VALUES` keyword, which is accompanied by a parentheses enclosed list of the values that correspond to each column name.

Important: Note that we *didn't specify* the "id" column name or value. Since we created the `cats` table with an "id" column whose type is `INTEGER PRIMARY KEY`, we don't have to specify the id column values when we insert data. Primary Key columns are auto-incrementing. As long as you have defined an id column with a data type of `INTEGER PRIMARY KEY`, a newly inserted row's id column will be automatically given the correct value.

Let's add a few more cats to our table. This time we'll do this via our text editor. Create a file, `01_insert_cats_into_cats_table.sql`. Use two `INSERT INTO` statements to insert the following cats into the table:

name	age	breed
"Lil' Bub"	5	"American Shorthair"
"Hannah"	1	"Tabby"

Each `INSERT INTO` statement gets its own line in the `.sql` file in your text editor. Each line needs to end with a `;`. Run the file with the following code in your terminal:

```
$ sqlite3 pets_database.db < 01_insert_cats_into_cats_table.sql
```

> **NOTE:** This command should be run from your terminal prompt, not in the `sqlite` console.

Now, we'll learn how to `SELECT` data from a table, which will help us to confirm that we inserted the above data correctly.

▶ *What immediately follows `INSERT INTO` in a SQL statement?*

Selecting Data

Now that we've inserted some data into our `cats` table, we likely want to read that data. This is where the `SELECT` statement comes in. We use it to retrieve database data, or rows.

Code Along 2: SELECT FROM

A basic `SELECT` statement works like this:

```
SELECT [names of columns we are going to select] FROM [table we are selecting from];
```

We specify the names of the columns we want to SELECT and then tell SQL the table we want to select them FROM.

We want to select all the rows in our table, and we want to return the data stored in any and all columns in those rows. To do this, we could pass the name of each column explicitly.

For the rest of this code along, you can run the SQL commands one of two ways, depending on your preference.

You can either open the database using the `sqlite3` CLI, and run the SQL commands from the terminal:

```
$ sqlite3 pets_database.db
```

Or you can open the `pets_database.db` file in DB Browser for SQLite, and run the SQL commands from the "Execute SQL" tab.

Run this command from the `sqlite` prompt in your terminal, or in DB Browser:

```
SELECT id, name, age, breed FROM cats;
```

This should give us back all the data from the `cats` table:

```
1|Maru|3|Scottish Fold
2|Lil' Bub|5|American Shorthair
3|Hannah|1|Tabby
```

A faster way to get data from every column in our table is to use a special selector, known commonly as the 'wildcard' selector `*`. The `*` selector means: "Give me all the data from all the columns for all of the cats" Using the wildcard, we can `SELECT` all the data from all of the columns in the cats table like this:

```
SELECT * FROM cats;
```

Now let's try out some more specific `SELECT` statements.

Selecting by Column Names

To select just certain columns from a table, use the following:

```
SELECT name FROM cats;
```

That should return the following:

```
Maru
Lil' Bub
Hannah
```

You can even select more than one column name at a time. For example, try out:

```
SELECT name, age FROM cats;
```

> **Top-Tip:** If you have duplicate data (for example, two cats with the same name) and you only want to select unique values, you can use the `DISTINCT` keyword. For example:

```
SELECT DISTINCT name FROM cats;
```

Selecting Based on Conditions: The `WHERE` Clause

What happens when we want to retrieve a specific table row? For example the row that belongs to Maru? Or to retrieve all the baby cats who are younger than two years old? We can use the `WHERE` keyword to select data based on specific conditions. Here's an example of a boilerplate `SELECT` statement using a `WHERE` clause.

```
SELECT * FROM [table name] WHERE [column name] = [some value];
```

Let's retrieve *just Maru* from our `cats` table:

```
SELECT * FROM cats WHERE name = "Maru";
```

That statement should return the following:

```
1|Maru|3|Scottish Fold
```

We can also use comparison operators, like `<` or `>` to select specific data. Let's give it a shot. Use the following statement to select the young cats:

```
SELECT * FROM cats WHERE age < 2;
```

The SQL statements we're learning here will eventually be used to integrate the applications you'll build with a database. For example, it's easy to imagine a web application that has many users. When a user signs into your app, you'll need to access your database and select the user that matches the credentials an individual is using to log in.

▶ *What immediately follows `SELECT` in a SQL statement?*

Updating Data

Let's talk about updating, or changing, data in our table rows. We do this with the `UPDATE` keyword.

Code Along 3: UPDATE

A boilerplate `UPDATE` statement looks like this:

```
UPDATE [table name] SET [column name] = [new value] WHERE [column name] = [value];
```

The `UPDATE` statement uses a `WHERE` clause to grab the row you want to update. It identifies the table name you are looking in and resets the data in a particular column to a new value.

Let's update one of our cats. Turns out Maru's friend Hannah is actually Maru's friend *Hana*. Let's update that row to change the name to the correct spelling:

```
UPDATE cats SET name = "Hana" WHERE name = "Hannah";
```

One last thing before we move on: deleting table rows.

Deleting Data

To delete table rows, we use the `DELETE` keyword.

Code Along 4: DELETE

A boilerplate `DELETE` statement looks like this:

```
DELETE FROM [table name] WHERE [column name] = [value];
```

Let's go ahead and delete Hana from our `cats` table (it turns out Hana is actually an iguana):

```
DELETE FROM cats WHERE id = 3;
```

Notice that this time we selected the row to delete using the Primary Key column. Remember that every table row has a Primary Key column that is unique. Hana was the third row in the database and thus had an id of `3`.

Conclusion

You've now successfully performed all four CRUD actions — Create, Read, Update, and Delete — using SQL with the `INSERT`, `SELECT`, `UPDATE` and `DESTROY` commands. These four actions are among the most important when it comes to working with SQL databases.

Resources

- **SQL Tutorial - W3Schools (https://www.w3schools.com/sql/)**
- **Documentation - SQLite (https://www.sqlite.org/docs.html)**
- **SQLite - VisualStudio Marketplace (https://marketplace.visualstudio.com/items?itemName=alexcvzz.vscode-sqlite)**
- **Datatypes in SQLite - SQlite (http://www.sqlite.org/datatype3.html)**
- **SQLite Keywords - SQLite (https://www.sqlite.org/lang_keywords.html)**
- **ZetCode sqlite3 Tutorial (http://zetcode.com/db/sqlite/)**